Frequent Itemset Mining: AprioriTID Algorithm

Nguyễn Đỗ Bảo — Student ID: [Your ID]

Ngày 4 tháng 4 năm 2026

1 Nền tảng lý thuyết

1.1 Bài toán khai thác tập phổ biến

1.1.1 Cơ sở dữ liệu giao dịch (transaction database)

Cơ sở dữ liệu giao dịch (CSDL) $\mathcal{D}$ bao gồm một tập hợp các giao dịch T ($\mathcal{D} = \{T_1, T_2, \ldots, T_n\}$), trong đó mỗi giao dịch chứa một tập hợp các hạng mục không rỗng (itemset) $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ sao cho $T \subseteq \mathcal{I}$ và được liên kết với một định danh giao dịch duy nhất (TID).

1.1.2 Độ hỗ trợ (support)

Độ hỗ trợ của một itemset X đo lường mức độ xuất hiện của X trong trong cơ sở dữ liệu giao dịch $\mathcal{D}$: $sup(X) = |\{T \mid X \subseteq T \wedge T \in \mathcal{D}\}|$. Có hai cách để biểu diễn độ hỗ trợ là độ hỗ trợ tuyệt đối (absolute support or support count) và độ hỗ trợ tương đối (relative support).

1.1.3 Tập phổ biến (frequent itemset)

Ta nói X là tập phổ biến khi $sup(X) \geq minsup$ trong đó $minsup$ là ngưỡng hỗ trợ tối thiểu. Một itemset chứa k item là một k-itemset. Tập hợp của các tập phổ biến k-itemset được ký hiệu là L_k.

1.1.4 Tập phổ biến đóng (closed frequent itemset) và tập phổ biến tối đại (maximal frequent itemset)

Itemset X được gọi là tập đóng (closed itemset) trong tập dữ liệu D nếu không tồn tại một siêu tập hợp (superitemset) Y nào chứa X mà Y có cùng số đếm hỗ trợ với X trong D: $\nexists Y \mid (X \subset Y) \wedge (\sup(X) = \sup(Y))$. Itemset X là tập phổ biến đóng (closed frequent itemset) khi thỏa mãn cả tính đóng và tính phổ biến.

Itemset X được gọi là tập tối đại (maximal itemset) trong tập dữ liệu D nếu không tồn tại một siêu tập hợp (superitemset) Y nào chứa X trong D: $\nexists Y \mid (X \subset Y)$. Itemset X là tập phổ biến tối đại (maximal frequent itemset) khi thỏa mãn cả tính tối đại và tính phổ biến.

Mối quan hệ giữa tập phổ biến (FI), tập phổ biến đóng (CFI), và tập phổ biến tối đại (MFI) được biểu diễn như sau: $MFI \subseteq CFI \subseteq FI$

1.1.5 Tính chất Apriori (downward closure)

Tính chất này phát biểu rằng tất cả các tập con không rỗng của một tập hạng mục phổ biến cũng bắt buộc phải là các tập phổ biến. Ngược lại, nếu một tập hạng mục không thỏa mãn điều kiện về độ hỗ trợ tối thiểu, thì không một tập cha nào của nó có khả năng thỏa mãn điều kiện đó. Bản chất đóng hướng xuống này cho phép các thuật toán khai phá cắt tỉa (prune) triệt để không gian tìm kiếm, loại bỏ một cách an toàn bất kỳ các mẫu ứng viên (candidate patterns) lớn hơn nào ngay khi phát hiện ra một tập con nhỏ hơn của nó là không phổ biến.

1.2 Phân tích thuật toán AprioriTID

Thuật toán tìm tập hạng mục lớn (large itemsets) hoạt động lặp đi lặp lại qua nhiều lượt quét CSDL. Ở lượt đầu tiên, thuật toán đếm độ hỗ trợ (support) của từng hạng mục để tìm các large 1-itemset. Ở các vòng tiếp theo, các large itemset từ vòng trước được dùng làm cơ sở sinh ra các tập ứng viên (candidate itemsets). Thuật toán tiếp tục quét CSDL để đếm support cho ứng viên, lọc ra các large itemset mới và lặp lại quá trình này cho đến khi không tìm thêm được tập nào nữa.

1.2.1 Ý tưởng cốt lõi

Thuật toán Apriori tìm tập phổ biến thông qua quá trình sinh ứng viên và kiểm tra theo từng mức độ dài, trong đó các large k-itemset được sử dụng để khám phá các large (k+1)-itemset. Ý tưởng cốt lõi của AprioriTID dựa trên một biến thể tối ưu hóa gọi là giảm thiểu giao dịch (transaction reduction). Nếu một giao dịch không chứa bất kỳ k-itemset phổ biến nào thì nó cũng không thể chứa bất kỳ (k+1)-itemset phổ biến nào. Do đó, ta loại bỏ các TID này và không cần xét đến chúng trong các lần quét CSDL ở các vòng lặp tiếp theo. Thay vì phải quét lại CSDL gốc nhiều lần, AprioriTID tạo ra một tập dữ liệu mã hóa trung gian, ký hiệu là $\overline{C}_k$ để lưu vết những ứng viên có mặt trong từng giao dịch. Ở các vòng lặp sau, tập $\overline{C}_k$ này có thể trở nên nhỏ hơn rất nhiều so với kích thước của CSDL gốc, giúp tiết kiệm đáng kể chi phí đọc đĩa (I/O).

Đầu tiên, tìm tập hợp 1-itemset phổ biến bằng cách đếm số lần xuất hiện của TID chứa 1-itemset, và thu thập những 1-itemset thỏa mãn minsup. Tập hợp kết quả được ký hiệu là L_1. Tiếp theo, L_1 được sử dụng để tìm L_2, tập hợp các 2-itemset phổ biến, được sử dụng để tìm L_3, và cứ thế tiếp tục cho đến khi không còn k-itemset phổ biến nào được tìm thấy nữa.

Các thuật toán đi trước như AIS hay SETM sinh các tập ứng viên trên đường bay (on-the-fly) bằng cách đọc từng giao dịch và mở rộng các tập phổ biến có trong giao dịch đó. Cách này tạo ra vô số ứng viên dư thừa, dẫn đến lãng phí tài nguyên. AprioriTID khắc phục điều này bằng cách sinh ứng viên một cách độc lập thông qua việc nối các tập phổ biến bước trước, tạo ra tập ứng viên nhỏ hơn rất nhiều.

1.2.2 Cấu trúc dữ liệu chính

AprioriTID sử dụng một cấu trúc dữ liệu mảng tuần tự đặc trưng là tập $\overline{C}_k$. Mỗi phần tử trong $\overline{C}_k$ đại diện cho một giao dịch, có định dạng: $\langle \text{TID}, \{X_k\} \rangle$, trong đó X_k là k-itemset

xuất hiện trong TID. Nếu một giao dịch không chứa bất kỳ ứng viên nào, nó sẽ không có mục (entry) tương ứng trong $\overline{C}_k$.

Lấy ví dụ minh họa CSDL $\mathcal{D}$ ở Hình 3 với mã giao dịch (TID) 101 chứa các mục (items): $\{1, 2, 3, 4\}$. Ngưỡng minsup = 2.

- Ở bước $k = 1$, $\bar{C}_1$ sẽ là: $\langle 101, \{\{1\}, \{2\}, \{3\}, \{4\}\}\rangle$.
- Ở bước $k = 2$, tập ứng viên C_2 chứa $\{1, 2\}, \{1, 4\}, \{1, 5\}, \ldots$ (lưu ý mục $\{3\}$ không đạt minsup ở L_1 nên các tập chứa 3 không xuất hiện trong C_2). Vì giao dịch 101 chứa các tổ hợp con hợp lệ nằm trong C_2 là $\{1, 2\}, \{1, 4\}, \{2, 4\}$, nên $\bar{C}_2$ cho TID 101 sẽ là: $\langle 101, \{\{1, 2\}, \{1, 4\}, \{2, 4\}\}\rangle$.
- Ở bước $k = 3$, tập ứng viên C_3 là $\{\{1, 2, 4\}, \{1, 2, 5\}, \{1, 4, 5\}, \{2, 4, 5\}\}$. Giao dịch 101 chỉ chứa tổ hợp $\{\{1, 2, 4\}\}$ có mặt trong C_3, nên $\bar{C}_3$ cho TID 101 sẽ là: $\langle 101, \{\{1, 2, 4\}\}\rangle$.
- Ở bước $k = 4$, tập ứng viên C_4 là $\{\{1, 2, 4, 5\}\}$. Giao dịch 101 không chứa đầy đủ các mục này, nên TID 101 sẽ bị loại bỏ hoàn toàn khỏi $\bar{C}_4$.

1.2.3 Mã giả (Pseudocode)

```
1)  L_1 = {các 1-itemset phổ biến};
2)  C̄_1 = cơ sở dữ liệu D;
3)  for (k = 2; L_{k-1} ≠ ∅; k++) do begin
4)      C_k = apriori-gen(L_{k-1});   // Tập ứng viên mới – xem Mục 2.1.1
5)      C̄_k = ∅;
6)      forall phần tử t ∈ C̄_{k-1} do begin
7)          // xác định các itemset ứng viên thuộc C_k có trong giao dịch mang mã định
            // danh t.TID
            C_t = {c ∈ C_k | (c − c[k]) ∈ t.set-of-itemsets ∧ (c − c[k − 1]) ∈ t.set-of-itemsets};
8)          forall ứng viên c ∈ C_t do
9)              c.count++;
10)         if (C_t ≠ ∅) then C̄_k += ⟨t.TID, C_t⟩;
11)     end
12)     L_k = {c ∈ C_k | c.count ≥ minsup}
13) end
14) Kết quả = ∪_k L_k;
```

Hình 1: Algorithm AprioriTid

Giải thích mã giả: 1) Tìm tập hợp các tập phổ biến có kích thước bằng 1 (L_1); 2) Khởi tạo $\overline{C}_1 = \mathcal{D}$. Mỗi item trong mỗi giao dịch được xem là một tập 1-itemset, tạo thành định dạng $\langle$TID, tập các 1-itemset$\rangle$; 3) Bắt đầu vòng lặp tìm các tập phổ biến kích thước k ($k \geq 2$). Vòng lặp này sẽ tiếp tục cho đến khi $L_{k-1} = \emptyset$; 4) Sinh ra tập các ứng viên kích thước k (C_k) từ tập phổ biến L_{k-1}. Hàm apriori-gen dùng để sinh ra tập các ứng viên k-itemset C_k từ tập phổ biến L_{k-1}; 5) Khởi tạo tập $\overline{C}_k$ rỗng; 6 - 11) Duyệt qua C_{k-1} để xác

định và tăng biến đếm độ hỗ trợ cho các tập ứng viên thực sự xuất hiện trong mỗi giao dịch. Sau đó tạo C_k chứa các giao dịch có chứa ít nhất một tập (k-1)-itemset phổ biến; 12) Lọc ra tập phổ biến L_k. Chỉ giữ lại những ứng viên trong C_k có count $\geq$ minsup; 14) Hợp nhất toàn bộ các tập L_k là danh sách tất cả các itemset phổ biến trong CSDL.

Hàm `apriori-gen` nhận đầu vào là L_{k-1}, tức là tập hợp tất cả các tập large (k-1)-itemset. Hàm trả về một tập hợp gồm các large k-itemset:

1. Đầu tiên, thực hiện hợp (join) tập L_{k-1} với chính nó để sinh ra các tập ứng viên C_k.
2. Tiếp theo, tỉa (prune) tất cả các tập hạng mục $c \in C_k$ nếu tồn tại bất kỳ (k-1)-itemset nào của c không nằm trong tập L_{k-1}.

1) **insert into** C_k
2) **select** $p.item_1, p.item_2, \ldots, p.item_{k-1}, q.item_{k-1}$
3) **from** $L_{k-1}\ p, L_{k-1}\ q$
4) **where** $p.item_1 = q.item_1, \ldots, p.item_{k-2} = q.item_{k-2}, p.item_{k-1} < q.item_{k-1}$;

5) **forall** itemsets $c \in C_k$ **do**
6) **forall** $(k-1)$-subsets s of c **do**
7) **if** $(s \notin L_{k-1})$ **then**
8) **delete** c from C_k;

Hình 2: Hàm apriori-gen

Lấy ví dụ CSDL $\mathcal{D}$ ở Hình 3 có L_2: {{1 2}, {1 4}, {1 5}, {2 4}, {4 5}}. Sau bước join, C_3: {{1 2 4}, {1 2 5}, {1 4 5}, {2 4 5}}. Bước prune xóa các itemset {{1 2 3}, {2 4 5}} vì các itemset {{2 3}, {2 5}} không nằm trong L_2. Do đó chỉ còn lại {{1 2 4}, {1 4 5}} ở C_3.

1.2.4 Phân tích độ phức tạp

Ký hiệu	**Ý nghĩa**
n	Số giao dịch trong CSDL $\mathcal{D}$
I	Số lượng items phân biệt
$k_{\max}$	Độ dài tập phổ biến dài nhất
$\lvert C_k \rvert$	Số ứng viên ở bước k
$\lvert L_k \rvert$	Số tập phổ biến ở bước k
w	Độ rộng trung bình của giao dịch
$\lvert \bar{C}_k \rvert$	Tổng số cặp $\langle \text{TID}, \text{itemset} \rangle$ trong $\bar{C}_k$

Ký hiệu tham số.

Bước 1 — Khởi tạo (dòng 1–2). Thuật toán quét toàn bộ CSDL $\mathcal{D}$ một lần duy nhất để tính độ hỗ trợ từng 1-itemset và khởi tạo $\bar{C}_1$. Độ phức tạp:

$$T_{\text{init}} = O(n \cdot w).$$

Bước 2 — Hàm `apriori-gen`: sinh ứng viên C_k. *Bước join (dòng 1–4, Hình 2).* Duyệt tất cả các cặp $(p, q) \in L_{k-1} \times L_{k-1}$; mỗi phép so sánh tốn $O(k)$ để kiểm tra $k-2$ item đầu giống nhau và item thứ $k-1$ theo thứ tự:

$$T_{\text{join}}(k) = O\Big(|L_{k-1}|^2 \cdot k\Big).$$

Bước prune (dòng 5–8, Hình 2). Với mỗi ứng viên $c \in C_k$, sinh k tập con kích thước $(k-1)$ rồi kiểm tra membership trong L_{k-1}:

$$T_{\text{prune}}(k) = O(|C_k| \cdot k \cdot |L_{k-1}|).$$

Tổng chi phí `apriori-gen` tại bước k:

$$T_{\text{gen}}(k) = O\Big(|L_{k-1}|^2 \cdot k \;+\; |C_k| \cdot k \cdot |L_{k-1}|\Big).$$

Bước 3 — Đếm support qua $\bar{C}_k$ (dòng 6–12, Hình 1). Đây là điểm then chốt phân biệt AprioriTID với Apriori cơ bản. Thay vì quét lại $\mathcal{D}$, thuật toán duyệt cấu trúc trung gian $\bar{C}_{k-1}$: với mỗi phần tử $t \in \bar{C}_{k-1}$, xác định tập ứng viên $C_t = \{c \in C_k \mid (c - c[k]) \in t.\text{set-of-itemsets} \;\wedge\; (c - c[k-1]) \in t.\text{set-of-itemsets}\}$, tăng $c.\text{count}$ cho mọi $c \in C_t$, rồi ghi $\langle t.\text{TID}, C_t\rangle$ vào $\bar{C}_k$ nếu $C_t \neq \emptyset$. Chi phí tại bước k phụ thuộc vào kích thước tổng thể $|\bar{C}_k|$:

$$T_{\text{count}}(k) = O\Big(|\bar{C}_k| \cdot k\Big).$$

Khi k tăng, $|\bar{C}_k|$ giảm nhanh do các giao dịch chứa ít ứng viên hơn, thường nhỏ hơn rất nhiều so với $n \cdot |C_k|$ — chi phí nếu quét lại $\mathcal{D}$.

Độ phức tạp thời gian tổng thể. Gộp cả ba bước:

$$\boxed{T_{\text{total}} = O(n \cdot w) + \sum_{k=2}^{k_{\max}} \Big[|L_{k-1}|^2 \cdot k \;+\; |C_k| \cdot k \cdot |L_{k-1}| \;+\; |\bar{C}_k| \cdot k\Big].}$$

Cấu trúc	**Không gian**	**Ghi chú**
CSDL gốc $\mathcal{D}$	$O(n \cdot w)$	Lưu trữ đầu vào
C_k, L_k mỗi vòng	$O(\lvert C_k\rvert \cdot k)$	Tập ứng viên và phổ biến
$\bar{C}_k$ (đặc trưng AprioriTID)	$O(n \cdot \lvert C_k\rvert)$	Lớn nhất ở $k = 2$, giảm dần

Độ phức tạp không gian. *Lưu ý:* $\bar{C}_k$ ở giai đoạn $k = 2$ có thể **lớn hơn** $\mathcal{D}$ gốc vì mỗi giao dịch ánh xạ tới nhiều cặp ứng viên. Đây là điểm yếu chính của AprioriTID về bộ nhớ so với Apriori tiêu chuẩn.

Phân tích trường hợp tốt / xấu.

- **Tốt nhất:** $|\bar{C}_k|$ giảm nhanh theo k (minsup cao, dữ liệu thưa). AprioriTID vượt trội nhờ tránh $k_{\max}$ lần quét I/O.
- **Xấu nhất:** Dữ liệu dày đặc, minsup thấp, $|\bar{C}_k|$ lớn ở mọi bước. Bộ nhớ bị đẩy lên cao; AprioriTID có thể chậm hơn Apriori cơ bản.
- **Tổng quát:** Độ phức tạp thời gian vẫn là *exponential* trong trường hợp xấu theo số items $|I|$, do số ứng viên tối đa là $O(2^{|I|})$.

1.2.5 So sánh lịch sử

Trong giai đoạn đầu của lĩnh vực khai phá tập phổ biến (FIM), các thuật toán tiên phong như AIS và SETM gặp nhiều hạn chế do phải sinh ứng viên "on-the-fly" (trên đường bay) trong quá trình đọc giao dịch và đòi hỏi việc rà quét, sắp xếp cơ sở dữ liệu gốc liên tục rất tốn kém. Sự ra đời của thuật toán Apriori và AprioriTID vào năm 1994 bởi R. Agrawal và R. Srikant đã tạo ra một bước ngoặt lớn khi giải quyết được những nhược điểm này. Điểm cải tiến mang tính lịch sử của AprioriTID là nó áp dụng hàm sinh ứng viên tiên nghiệm giúp thu hẹp không gian tìm kiếm, đồng thời sử dụng cấu trúc dữ liệu trung gian mã hóa ($\overline{C}_k$) để đếm độ hỗ trợ. Nhờ vậy, thuật toán không cần phải quét lại cơ sở dữ liệu gốc từ vòng lặp thứ hai trở đi, giúp giảm thiểu đáng kể chi phí truy xuất đĩa (I/O) so với các thuật toán tiền nhiệm.

Mặc dù là một bước tiến lớn, AprioriTID vẫn để lại một hạn chế cốt lõi: nó vẫn hoàn toàn phụ thuộc vào mô hình "sinh và kiểm tra" (generate-and-test), sinh ra vô số ứng viên nhỏ không cần thiết và dễ gây quá tải bộ nhớ khi xử lý các tập dữ liệu lớn. Để giải quyết bài toán này, các thế hệ thuật toán FIM sau đó đã thay đổi hoàn toàn cách tiếp cận. Nổi bật là thuật toán FP-Growth ra đời bằng cách áp dụng cấu trúc cây nén FP-tree kết hợp với chiến lược "chia để trị", giúp khai phá trực tiếp các tập phổ biến mà hoàn toàn không cần phải sinh ứng viên. Song song đó, thuật toán Eclat cũng xuất hiện, giải quyết điểm nghẽn bằng cách chuyển sang sử dụng định dạng dữ liệu dọc (vertical data format), cho phép tính toán độ hỗ trợ cực kỳ nhanh chóng và hiệu quả thông qua phép giao các tập mã giao dịch (TID_set).

2 Ví dụ minh họa

2.1 Ví dụ 1: Cơ sở

Tập itemset phổ biến từ CSDL ở Hình 3 là {{1}, {2}, {4}, {5}, {1 2}, {1 4}, {1 5}, {2 4}, {4 5}, {1 2 4}, {1 4 5}}

2.2 Ví dụ 2: Tình huống đặc biệt

Database

TID	Items
101	1 2 3 4
102	1 2
103	1 4 5
104	1 2 4 5
105	5

$\overline{C}_1$

TID	Tập Itemset
101	{{1}, {2}, {3}, {4}}
102	{{1}, {2}}
103	{{1}, {4}, {5}}
104	{{1}, {2}, {4}, {5}}
105	{5}

L_1

Itemset	Support
{1}	4
{2}	3
{4}	3
{5}	3

C_2

Itemset
{1 2}
{1 4}
{1 5}
{2 4}
{2 5}
{4 5}

$\overline{C}_2$

TID	Tập Itemset
101	{{1 2}, {1 4}, {2 4}}
102	{{1 2}}
103	{{1 3}, {1 5}}
104	{{1 2}, {1 5}, {2 5}}

L_2

Itemset	Support
{1 2}	3
{1 4}	3
{1 5}	2
{2 4}	2
{4 5}	2

C_3

Itemset
{1 2 4}
{1 2 5}
{1 4 5}
{2 4 5}

$\overline{C}_3$

TID	Tập Itemset
101	{{1 2 4}}
103	{{1 4 5}}
104	{{1 2 4}, {1 2 5}, {1 4 5}, {2 4 5}}

L_3

Itemset	Support
{1 2 4}	2
{1 4 5}	2

C_4

Itemset
{1 2 4 5}

$\overline{C}_4$

TID	Tập Itemset
104	{1 2 4 5}

L_4

Itemset	Support

Hình 3: Ví dụ minh họa